Bonus Topic: Containers

Contents

Part 1: Introduction and Motivation

The “Works on My Machine” Problem

Traditional deployment challenges:

- Application runs on developer’s machine → fails in production
- Different dependency versions cause conflicts
- Different OS configurations create surprises
- Slow, error-prone manual deployment

Solution: Containers

- Package application + dependencies + runtime
- Consistent environment across all stages
- Lightweight and fast to deploy

Containers vs Virtual Machines

Aspect	Containers	VMs
Startup	Seconds	Minutes
Resource	Minimal	Significant
Density	50+	3-10
Isolation	Process-level	Hardware-level
Performance	Near-native	Slight overhead

Key Insight: Containers share the host OS kernel; VMs include full guest OS

Part 2: Theoretical Foundations

Core Concept: Process Isolation

What is a container?

- An isolated environment where processes run
- Uses OS mechanisms to create isolation
- Shares the underlying kernel

Not new technology!

- Unix has had process isolation capabilities for decades
- Containers combine existing OS features in a clever way
- Introduced at scale by Docker (2013)

Linux Namespaces: Overview

Namespaces partition system resources so processes in different namespaces cannot see each other's resources.

Six main types:

1. PID Namespace
2. Network Namespace
3. Mount Namespace
4. IPC Namespace
5. User Namespace
6. Unix Timesharing System (UTS) Namespace

```
man unshare
```

PID Namespace

Process ID Isolation

- Each namespace has its own process tree
- Container sees itself as PID 1 (init process)
- Host sees container processes with different PIDs
- Enables independent process management

Example:

```
Container view: PID 1, 2, 3 (my processes)
Host view: PID 5432, 5433, 5434 (same processes)
```

Network Namespace

Network Isolation

- Each namespace has own network interfaces
- Own routing tables and firewall rules
- Own IP addresses and port space
- Can both listen on port 8080 without conflict

Networking architectures:

- Bridge: containers connected via virtual switch
- Host: share host’s network (no isolation)
- Overlay: for multi-host communication

Mount Namespace

Filesystem Hierarchy Isolation

- Each namespace has independent filesystem hierarchy
- Container has its own `/`, `/usr`, `/home`
- Don’t affect host filesystem
- Can mount different filesystems

Enables:

- Complete filesystem isolation
- Different OS distributions in containers
- Independent mount points

IPC Namespace

Inter-Process Communication Isolation

- Isolates System V IPC objects
- Isolates POSIX message queues
- Prevents containers interfering with each other
- Important for security and stability

User Namespace

User ID Mapping

- Maps user IDs in namespace to host UIDs
- Root in container = unprivileged user on host
- **Critical security feature**
- Prevents privilege escalation

Security benefit:

- Container root ≠ host root
- Even if container is compromised, attacker has limited privileges

UTS Namespace

Hostname Isolation

- Each container has its own hostname
- Each container has its own domain name
- Small but important for configuration
- Processes see different `hostname` command output

```
#view uts namespaces
sudo lsns | grep uts

#create a uts namespace, attach bash to it
#a new shell will appear where you can change the hostname
sudo unshare --uts /bin/bash
```

Control Groups (cgroups)

Resource Limits and Accounting

Namespaces provide isolation; cgroups provide resource enforcement

Key cgroup features:

- CPU limiting
- Memory limiting
- I/O limiting
- Device access control
- Process freezing

cgroups: CPU and Memory

CPU Limiting

- Restrict CPU time allocation
- Measured in shares or absolute quotas
- Prevents resource hogging

Memory Limiting

- Hard limits on memory usage
- OOM killer if exceeded
- Soft limits with pressure notifications
- Memory accounting across containers

cgroups: I/O and Device Control

I/O Limiting

- Restrict disk bandwidth
- Per-device throttling
- Prevents “noisy neighbor” problem

Device Access Control

- Specify allowed block/character devices
- Container might only access `/dev/null`, `/dev/zero`, storage
- Fine-grained security control

Union Filesystems & Layering

Efficient Storage through Layers

- Container images consist of stacked read-only layers
- Top layer is read-write at runtime
- Layers are deduplicated and compressed

Copy-on-Write (CoW)

- When writing to lower-layer file, copy to top layer
- Multiple containers share base layers efficiently
- Only modifications consume storage

Layering Example

```
Layer 1: Ubuntu base OS        (500 MB)
Layer 2: Python runtime        (50 MB)
Layer 3: Application code      (10 MB)
--------------------------------------
Container 1 runtime           (5 MB)
Container 2 runtime           (3 MB)

Total: 568 MB (not 1.136 GB!)
```

Container Security Model

Important: Containers ≠ VMs in terms of isolation

Containers share kernel → kernel vulnerability affects all

Security layers:

1. Namespaces (basic isolation)
2. User namespaces (unprivileged root)
3. Capabilities (restrict privileged operations)
4. SELinux/AppArmor (Mandatory Access Control)
5. Seccomp filters (restrict syscalls)
6. Read-only filesystems
7. Non-root user enforcement

Part 3: Implementation Details

Container Lifecycle

Creating and Running a Container

```
1. Create namespaces
2. Set resource limits (cgroups)
3. Drop unnecessary capabilities
4. Change root (chroot/pivot_root)
5. Execute container process
```

Process runs as PID 1 in isolated environment

Creating a Container: Pseudocode

```
// Set up isolation
clone_flags = CLONE_NEWPID | CLONE_NEWNET |
              CLONE_NEWNS | CLONE_NEWIPC |
              CLONE_NEWUTS | CLONE_NEWUSER;
// Fork with namespaces
child_pid = clone(container_main, stack,
                  clone_flags, args);
// Configure resource limits
configure_cgroups(child_pid);    // Memory: 512MB, CPU: 1 core
set_uid_mapping(child_pid);      // Root mapping
mount_container_filesystem(child_pid);  // Pivot to container root
// Wait for container
waitpid(child_pid);
```

Example: https://github.com/srg-ics-uplb/simple_container

Container Images

Image Structure

- TAR archives of filesystem layers
- Metadata: environment, entry point, ports
- Manifest: describes layers and relationships

Content-Addressable

- Each layer has SHA256 digest
- Enables deduplication
- Two images sharing a layer → only one copy

Image Registry

- Central repository (Docker Hub, quay.io)
- Enables sharing and distribution
- Private registries for enterprises

Building Container Images

Dockerfile/Containerfile

- Declarative recipe for building images
- Each instruction creates a new layer

```
################################################################
# ICS-OS Development
################################################################
FROM ubuntu:16.04
RUN apt-get update
RUN apt-get install -y build-essential nasm qemu-kvm tcc git \\
        gcc-multilib sudo
RUN mkdir -p /home/ics-os
```

Build cache

- Layers cached for efficiency
- Rebuilds only changed layers

Container Runtimes

OCI Standard (Open Container Initiative)

- Standardizes container format and behavior
- Enables interoperability

Common Runtimes

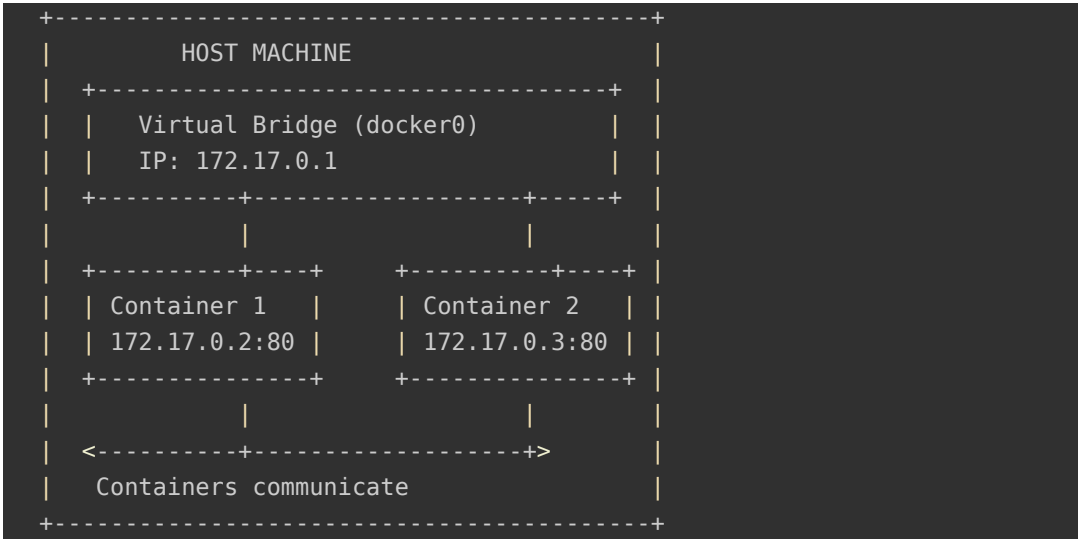
- `runc`: Reference OCI implementation
- `containerd`: Lightweight core runtime
- `cri-o`: For Kubernetes

Separation of Concerns

- Daemon: manages multiple containers (dockerd, podman)
- Runtime: handles single container lifecycle (runc)

Networking: Bridge Network

Default for many setups



Networking Modes

Bridge Network

- Default, good isolation
- Containers on same bridge communicate
- Outside traffic via NAT

Host Network

- Container uses host’s network namespace
- No isolation, maximum performance
- Used for high-performance apps

Overlay Network

- Multi-host communication
- Uses VXLAN or similar tunneling
- For orchestration systems

Volume Mounting and Storage

Three Types of Mounts

1. **Bind Mounts**: Host directory → container
   - Changes visible both sides
   - Used for development
2. **Volumes**: Runtime-managed storage
   - Portable, survives container deletion
   - Different backends available
3. **tmpfs Mounts**: In-memory storage
   - Fast, temporary
   - Cleared on container stop

Part 4: Orchestration and Scale

Why Orchestration?

Single container is simple Managing many is complex

Challenges:

- Container placement on machines
- Resource scheduling and bin packing
- Service discovery and networking
- Rolling updates and zero-downtime deployments
- Health monitoring and self-healing
- Auto-scaling based on load

Container Orchestration Platforms

Kubernetes

- Most popular, feature-rich
- Declarative desired state
- Self-healing, auto-scaling
- Learning curve but powerful

Docker Swarm

- Simpler, built into Docker
- Good for smaller deployments

Managed Services

- Amazon ECS
- Azure Container Instances
- Cloud provider handles infrastructure

Part 5: Comparison and Trade-offs

Containers vs Virtual Machines

Feature	Containers	VMs
Startup	Seconds	Minutes
Memory	Minimal	Hundreds of MB
Density	50-100+	3-10
Isolation	Process	Hardware
Security	Good	Excellent
Overhead	Minimal	Significant

Rule of thumb: Containers for cloud-native, VMs for maximum isolation

When to Use Containers

- Microservices architecture
- Rapid deployment cycles
- Cloud-native applications
- High density workloads
- Development-production parity

When VMs Might Be Better

- Maximum security isolation required
- Legacy applications
- Mixed OS environments
- Kernel customization needed
- Strict compliance requirements

Part 6: Live Demonstration

Demo Outline

```
# 1. List containers and images
docker ps
docker images

# 2. Run a container interactively
docker run -it --name demo ubuntu /bin/bash
  ps aux        # Different from host!
  exit

# 3. Resource limits in action
docker run --memory 256m nginx

# 4. Mount namespaces
docker run -v /host:/container alpine ls

# 5. Inspect details
docker inspect <container>
```

Demo Key Takeaways

1. **Containers are real processes**
   - Visible on host with different PIDs
   - Can use standard Unix tools
2. **Namespaces create isolation illusion**
   - Container sees itself as PID 1
   - Isolated filesystem and network
3. **cgroups enforce resource limits**
   - Memory, CPU, I/O restrictions
   - Measurable resource usage
4. **Union filesystems enable efficiency**
   - Layer reuse across containers
   - Significant storage savings

References

- OCI Standards: https://opencontainers.org/
- Linux Namespaces: `man namespaces`, `man clone`
- cgroups Documentation: Linux kernel docs
- Docker Documentation: https://docs.docker.com/
- Kubernetes Documentation: https://kubernetes.io/docs/